2 *How contract testing fits into wider testing concepts*

This chapter covers

- Contract testing and other testing types
- Whether contract testing can replace other types of tests
- Benefits of contract testing
- Introducing contract testing to your teams
- Challenges with contract testing
- Practical guidance on introducing contract testing to projects

Contract testing is unique when compared to other testing types because it requires developing two sets of tests to fully reap the benefits. A consumer test can exist without a provider test, but it won't be useful. You can initiate the contract testing process by writing either a consumer test or a provider test individually, but it's important to note that these individual tests alone don't give you real benefits. To unlock the full potential of contract testing, you must implement and execute both consumer and provider tests in tandem, in a continuous integration (CI) pipeline ideally.

Let's consider an example to highlight the problem of only writing either consumer or provider tests. In a previous project undertaken by one of the authors, there was a web application communicating to a GraphQL service. This GraphQL layer was responsible for communicating with various data services, both legacy and brand new. In this scenario, contract testing was introduced between the GraphQL layer and the data services. The GraphQL layer played the role of the consumer, while the different data services served as the providers. During the initial implementation of contract testing, only the consumer tests were developed within the GraphQL codebase, as illustrated in figure 2.1. The consumer is generating the contract, but it's not being verified.